Local search with reusing moves

Hubert Nowakowski 160302

Muhammad Sattorov 159351

Code link: https://github.com/hnowakowski/Evolutionary-Computation-2026/blob/main/5_reuse/main.cpp

Description of the problem

We're trying to find a hamiltonian cycle where:

- function to be minimized is defined as distance + cost
- we want to have the cycle consist of exactly 50% of nodes
- distances between nodes are rounded euclidean

All solutions are described as a vector of subsequent node indices, their objectives were tested using the solution checker excel sheet

For random solutions the std::mt19937 random number generator was used

Psuedocode

Definition of a move

Since we're keeping both types of moves in one pool, we need them to be directly comparable.

We're also storing info about original successors and predecessors for node swaps, so that it's easier to check if a move is still valid.

Delta is only ever calculated when creating a move object ie. when discovering a new improving move and adding it into the pool.

Delta is the comparison operator for the minheap.

```
struct Move:
    bool isEdge
    # for edges it's just i, j, k, l indices
    # for nodes it's 0: old, 1: new, 2: old predecessor, 3: old successor
    array[4] swap
    int delta

    node swap constructor:
        iIdx = solLookup[i]
        beforeI = (iIdx == 0) ? sol.size() - 1 : iIdx - 1
        afterI = (iIdx == sol.size() - 1) ? 0 : iIdx + 1
```

```
        swap = {i, j, sol[beforeI], sol[afterI]}
        delta = interNodeSwapDelta(matrix, sol, nodes, solLookup, i, j)
        isEdge = false

    edge swap contructor:
        swap = { i, j, k, l }
        delta = edgeSwapDelta(matrix, sol, i, j, k, l)
        isEdge = true
```

Helper for edge direction

Since the direction of an edge `i - j` is a frequently brought up issue, we have a small helper function.

In short, it returns 1 if the edge is its original state, -1 if it's reversed, and 0 if it does not exist anymore.

```
edgeState:
    idxI = solLookup[i]
    idxJ = solLookup[j]
    if idxI == -1 || idxJ == -1: return 0
    afterI = (idxI == sol.size() - 1) ? 0 : idxI + 1
    afterJ = (idxJ == sol.size() - 1) ? 0 : idxJ + 1
    if idxJ == afterI: return 1
    if idxI == afterJ: return -1
    return 0
```

Algorithm

```
sol = initial_algorithm()
array[200] solLookup
fill solLookup with -1
for i in range(0, sol.size):
    solLookup[sol[i]] = i

priority_queue<Move> movePool
for i in range(0, sol.size):
    for j in range(0, 200):
        if (solLookup[j] != -1) continue
            m = Move(sol[i], j)
            if (m.delta < 0) movePool.emplace(m)

for i in range(0, sol.size()):
    j = (i == sol.size() - 1) ? 0 : i + 1
    for k in range(i + 2, sol.size()):
        l = (k == sol.size() - 1) ? 0 : k + 1
        if (i == 0 && l == 0) continue
        m = Move(sol[i], sol[j], sol[k], sol[l])
        if (m.delta < 0) movePool.emplace(m)

vector<Move> reservePool
```

```
while true:
    foundUsable = false
    while !movePool.empty:
        m = movePool.top
        i, j, k, l = m.swap
        if !m.isEdge:
            # nodes missing in sol (or j present)
            if solLookup[i] == -1 || solLookup[j] != -1 || solLookup[k] == -1 ||
solLookup[l] == -1:
                movePool.pop()
                continue

            # adjacent edges missing
            beforeI = (solLookup[i] == 0) ? sol.size() - 1 : solLookup[i] - 1
            afterI = (solLookup[i] == sol.size() - 1) ? 0 : solLookup[i] + 1
            if sol[beforeI] != k || sol[afterI] != l:
                movePool.pop()
                continue

            update solLookup
            movePool.pop()
            foundUsable = true
            break
        else:
            state1 = edgeState(i, j)
            state2 = edgeState(k, l)
            # edges missing in sol
            if state1 == 0 || state2 == 0:
                movePool.pop()
                continue;

            # one edge reversed one not
            if state1 != state2:
                movePool.pop()
                reservePool.emplace_back(m)
                continue

            if state1 == 1:
                sol.reverse(solLookup[j], solLookup[k])
                update solLookup
            else:
                sol.reverse(solLookup[k], solLookup[j])
                update solLookup
            movePool.pop()
            foundUsable = true
            break

    if foundUsable:
        for each m in reservePool:
            movePool.emplace(m)
        reservePool.clear()

    if !foundUsable:
        reservePool.clear()
```

```
        fill movePool as described above the loop

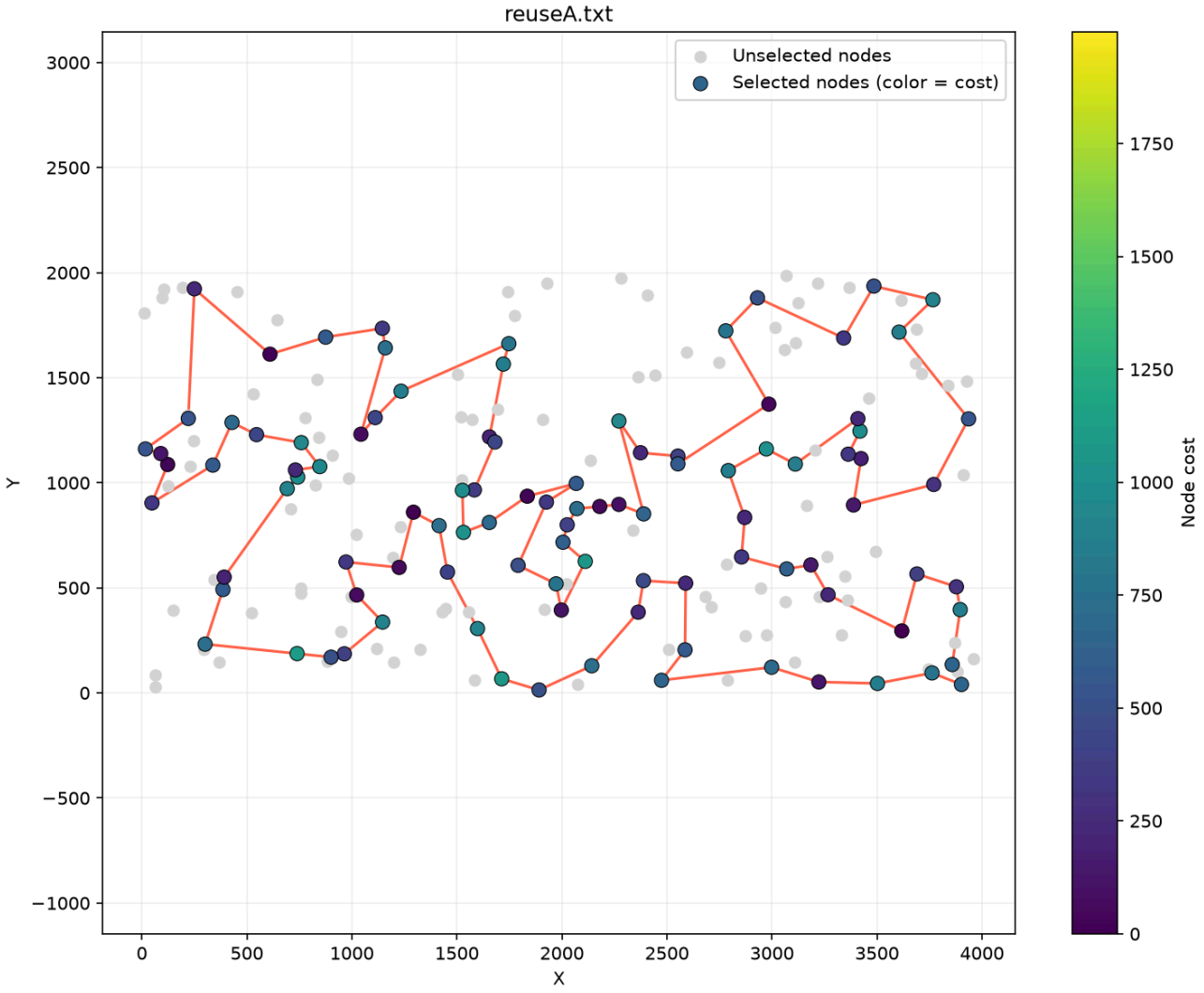
    if movePool.empty: break

return solution
```

Scores

TSPA

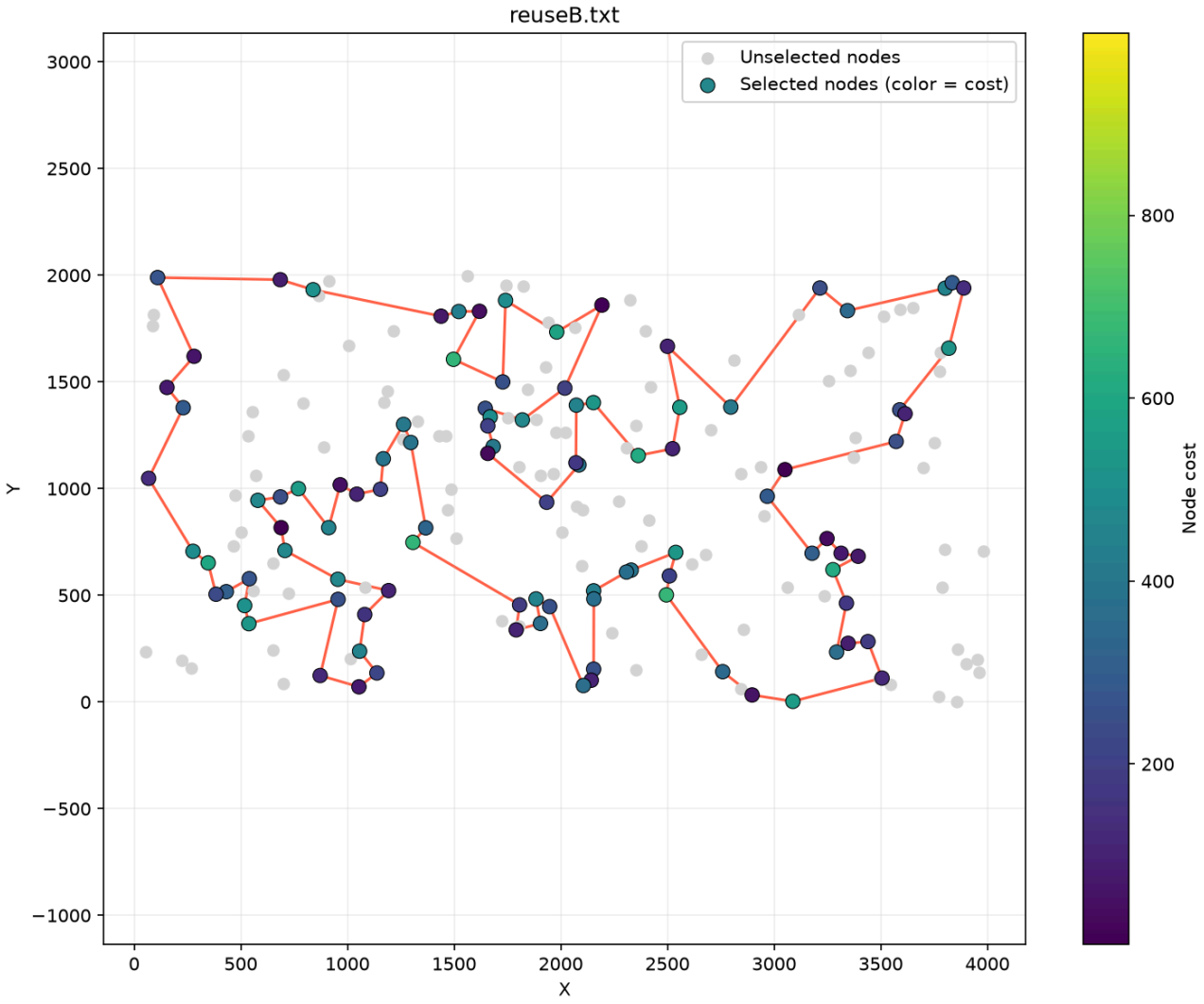
Max objective: 87957
Average objective: 76747
Min objective: 74358
Elapsed: 1.35825 seconds




```
78 88 64 57 182 192 158 134 84 58 181 114 25 98 87 28 180 141 30 187 189 2 171 92
69 127 97 160 83 123 90 100 152 108 49 168 63 149 116 60 48 80 91 157 71 132 76 4
112 139 51 16 35 156 102 185 67 52 144 45 73 135 41 140 53 179 177 36 131 143 33 9
54 164 186 136 178 29 55 101 147 47 93 50 3 169 1 85 142 8 174 176 150 106 89 107
130 159 170 153
```

TSPB

Max objective: 55660
Average objective: 52871
Min objective: 50862
Elapsed: 1.29707 seconds




```
117 133 136 172 160 69 151 3 184 46 86 135 76 185 57 104 190 70 81 94 105 61 196
114 10 54 192 80 127 129 90 52 153 74 20 14 2 17 171 99 31 67 66 24 34 152 181 179
87 82 167 73 51 85 59 33 64 21 30 38 47 140 154 97 26 125 177 170 115 155 40 60
157 45 134 175 71 36 197 6 29 132 103 178 65 145 28 149 37 116 147 121 198 96 63
120 48 156 89 72
```

Scores

Time is the full execution time in seconds of 200 runs on a single thread.

Algo name	average TSPA (min - max)	TSPA time	average TSPB (min - max)	TSPB time
Local search 5	90444 (85663 - 96792)	1.97878	55955 (52900 - 59453)	1.97842

Algo name	average TSPA (min - max)	TSPA time	average TSPB (min - max)	TSPB time
Local search 10	81890 (77930 - 86959)	2.67813	53767 (51171 - 56922)	2.76157
Local search 20	77937 (75343 - 81108)	4.20468	53305 (51171 - 56060)	4.51270
Local search reuse	76747 (74358 - 87957)	**1.35825**	52871 (50862 - 55660)	**1.29707**
Local search full	**76344** (**73968** - **79004**)	4.77540	**52345** (**50554** - **54536**)	4.95898

Conclusions

By reusing moves, we were able to vastly decrease the running time of the algorithm, almost by 4 times, while only marginally decreasing scores.

In every instance, the reusing mechanism yielded much better results AND runtime than the candidates approach (aside from max score in TSPA).

For TSPB the difference between full and reusing approach is smaller, but for TSPA it's pretty decent as well.